

connections between the problem of computing marginals and that of computing modes.

To understand the challenges inherent in these inference problems, consider the case of a discrete random vector $x \in \mathcal{X}^m$, where $\mathcal{X}_s = \{0, 1, \dots, r - 1\}$ for each vertex $s \in V$. A naive approach to computing a marginal at a single node — say $p(x_s)$ — entails summing over all configurations of the form $\{x' \in \mathcal{X}^m \mid x'_s = x_s\}$. Since this set has r^{m-1} elements, it is clear that a brute force approach will rapidly become intractable. Even with binary variables ($r = 2$) and a graph with $m \approx 100$ nodes (a small size for many applications), this summation is beyond the reach of brute-force computation. Similarly, in this discrete case, computing a mode entails solving an integer programming problem over an exponential number of configurations. For continuous random vectors, the problems are no easier¹ and typically harder, since they require computing a large number of integrals.

For undirected graphs without cycles or for directed graphs in which each node has a single parent — known generically as *trees* in either case — these inference problems can be solved exactly by recursive “message-passing” algorithms of a dynamic programming nature, with a computational complexity that scales only linearly in the number of nodes. In particular, for the case of computing marginals, the dynamic programming solution takes the form of a general algorithm known as the *sum-product algorithm*, whereas for the problem of computing modes it takes the form of an analogous algorithm known as the *max-product algorithm*. We describe these algorithms in Section 2.5.1. More generally, as we discuss in Section 2.5.2, the *junction tree algorithm* provides a solution to inference problems for arbitrary graphs. The junction tree algorithm has a computational complexity that is exponential in a quantity known as the *treewidth* of the graph.

2.4 Applications

Before turning to these algorithmic issues, however, it is helpful to ground the discussion by considering some examples of applications of

¹The Gaussian case is an important exception to this statement.

graphical models. We present examples of the use of graphical models in the general areas of Bayesian hierarchical modeling, contingency table analysis, and combinatorial optimization and satisfiability, as well specific examples in bioinformatics, speech and language processing, image processing, spatial statistics, communication and coding theory.

2.4.1 Hierarchical Bayesian Models

The Bayesian framework treats all model quantities — observed data, latent variables, parameters, nuisance variables — as random variables. Thus, in a graphical model representation of a Bayesian model, all such variables appear explicitly as vertices in the graph. The general computational machinery associated with graphical models applies directly to Bayesian computations of quantities such as marginal likelihoods and posterior probabilities of parameters. Although Bayesian models can be represented using either directed or undirected graphs, it is the directed formalism that is most commonly encountered in practice. In particular, in hierarchical Bayesian models, the specification of prior distributions generally involves additional parameters known as hyperparameters, and the overall model is specified as a set of conditional probabilities linking hyperparameters, parameters and data. Taking the product of such conditional probability distributions defines the joint probability; this factorization is simply a particular instance of Equation (2.1).

There are several advantages to treating a hierarchical Bayesian model as a directed graphical model. First, hierarchical models are often specified by making various assertions of conditional independence. These assertions imply other conditional independence relationships, and the reachability algorithms (mentioned in Section 2.1) provide a systematic method for investigating all such relationships. Second, the visualization provided by the graph can be useful both for understanding the model (including the basic step of verifying that the graph is acyclic), as well for exploring extensions. Finally, general computational methods such as MCMC and variational inference algorithms can be implemented for general graphical models, and hence apply to hierarchical models in graphical form. These advantages and others have

led to the development of general software programs for specifying and manipulating hierarchical Bayesian models via the directed graphical model formalism [93].

2.4.2 Contingency Table Analysis

Contingency tables are a central tool in categorical data analysis [1, 80, 151], dating back to the seminal work of Pearson, Yule, and Fisher. An m -dimensional contingency table with r levels describes a probability distribution over m random variables $(X_1, \dots, X_m)$, each of which takes one of r possible values. For the special case $m = 2$, a contingency table with r levels is simply a $r \times r$ matrix of nonnegative elements summing to one, whereas for $m > 2$, it is a multi-dimensional array with r^m nonnegative entries summing to one. Thus, the table fully specifies a probability distribution over a random vector $(X_1, \dots, X_m)$, where each X_s takes one of r possible values.

Contingency tables can be modeled within the framework of graphical models, and graphs provide a useful visualization of many natural questions. For instance, one central question in contingency table analysis is distinguishing different orders of interaction in the data. As a concrete example, given $m = 3$ variables, a simple question is testing whether or not the three variables are independent. From a graphical model perspective, this test amounts to distinguishing whether the associated interaction graph is completely disconnected or not (see panels (a) and (b) in Figure 2.3). A more subtle test is to distinguish whether the random variables interact in only a pairwise manner (i.e., with factors ψ_{12} , ψ_{13} , and ψ_{23} in Equation (2.2)), or if there is actually a three-way interaction (i.e., a term ψ_{123} in the factorization (2.2)). Interestingly, these two factorization choices cannot be distinguished

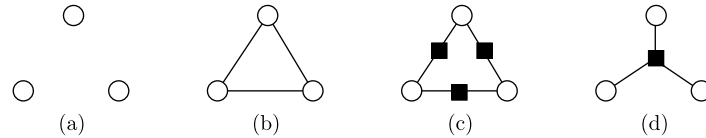


Fig. 2.3 Some simple graphical interactions in contingency table analysis. (a) Independence model. (b) General dependent model. (c) Pairwise interactions only. (d) Triplet interactions.

using a standard undirected graph; as illustrated in Figure 2.3(b), the fully connected graph on 3 vertices does not distinguish between pairwise interactions without triplet interactions, versus triplet interaction (possibly including pairwise interaction as well). In this sense, the factor graph formalism is more expressive, since it can distinguish between pairwise and triplet interactions, as shown in panels (c) and (d) of Figure 2.3.

2.4.3 Constraint Satisfaction and Combinatorial Optimization

Problems of constraint satisfaction and combinatorial optimization arise in a wide variety of areas, among them artificial intelligence [66, 192], communication theory [87], computational complexity theory [55], statistical image processing [89], and bioinformatics [194]. Many problems in satisfiability and combinatorial optimization [180] are defined in graph-theoretic terms, and thus are naturally recast in the graphical model formalism.

Let us illustrate by considering what is perhaps the best-known example of satisfiability, namely the 3-SAT problem [55]. It is naturally described in terms of a factor graph, and a collection of binary random variables $(X_1, X_2, \dots, X_m) \in \{0, 1\}^m$. For a given triplet $\{s, t, u\}$ of vertices, let us specify some “forbidden” configuration $(z_s, z_t, z_u) \in \{0, 1\}^3$, and then define a triplet compatibility function

$$\psi_{stu}(x_s, x_t, x_u) = \begin{cases} 0 & \text{if } (x_s, x_t, x_u) = (z_s, z_t, z_u), \\ 1 & \text{otherwise.} \end{cases} \quad (2.3)$$

Each such compatibility function is referred to as a *clause* in the satisfiability literature, where such compatibility functions are typically encoded in terms of logical operations. For instance, if $(z_s, z_t, z_u) = (0, 0, 0)$, then the function (2.3) can be written compactly as $\psi_{stu}(x_s, x_t, x_u) = x_s \vee x_t \vee x_u$, where $\vee$ denotes the logical OR operation between two Boolean symbols.

As a graphical model, a single clause corresponds to the model in Figure 2.3(d); of greater interest are models over larger collections of binary variables, involving many clauses. One basic problem in

satisfiability is to determine whether a given set of clauses F are satisfiable, meaning that there exists some configuration $x \in \{0, 1\}^m$ such that $\prod_{(s,t,u) \in F} \psi_{stu}(x_s, x_t, x_u) = 1$. In this case, the factorization

$$p(x) = \frac{1}{Z} \prod_{(s,t,u) \in F} \psi_{stu}(x_s, x_t, x_u)$$

defines the uniform distribution over the set of satisfying assignments.

When instances of 3-SAT are randomly constructed — for instance, by fixing a clause density $\alpha > 0$, and drawing $\lceil \alpha m \rceil$ clauses over m variables uniformly from all triplets — the graphs tend to have a locally “tree-like” structure; see the factor graph in Figure 2.9(b) for an illustration. One major problem is determining the value α^* at which this ensemble is conjectured to undergo a phase transition from being satisfiable (for small α , hence with few constraints) to being unsatisfiable (for sufficiently large α). The survey propagation algorithm, developed in the statistical physics community [170, 171], is a promising method for solving random instances of satisfiability problems. Survey propagation turns out to be an instance of the sum-product or belief propagation algorithm, but as applied to an alternative graphical model for satisfiability problems [41, 162].

2.4.4 Bioinformatics

Many classical models in bioinformatics are instances of graphical models, and the associated framework is often exploited in designing new models. In this section, we briefly review some instances of graphical models in both bioinformatics, both classical and recent. Sequential data play a central role in bioinformatics, and the workhorse underlying the modeling of sequential data is the *hidden Markov model* (HMM) shown in Figure 2.4(a). The HMM is in essence a dynamical version of a finite mixture model, in which observations are generated conditionally on a underlying latent (“hidden”) state variable. The state variables, which are generally taken to be discrete random variables following a multinomial distribution, form a Markov chain. The graphical model in Figure 2.4(a) is also a representation of the state-space model underlying Kalman filtering and smoothing, where the state variable is a

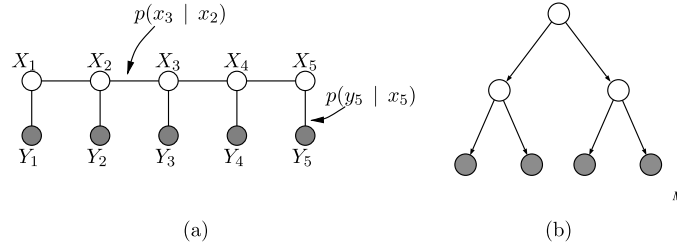


Fig. 2.4 (a) The graphical model representation of a generic hidden Markov model. The shaded nodes $\{Y_1, \dots, Y_5\}$ are the observations and the unshaded nodes $\{X_1, \dots, X_5\}$ are the hidden state variables. The latter form a Markov chain, in which X_s is independent of X_u conditional on X_t , where $s < t < u$. (b) The graphical model representation of a phylogeny on four extant organisms and M sites. The tree encodes the assumption that there is a first speciation event and then two further speciation events that lead to the four extant organisms. The box around the tree (a “plate”) is a graphical model representation of replication, here representing the assumption that the M sites evolve independently.

Gaussian vector. These models thus also have a right to be referred to as “hidden Markov models,” but the terminology is most commonly used to refer to models in which the state variables are discrete.

Applying the junction tree formalism to the HMM yields an algorithm that passes messages in both directions along the chain of state variables, and computes the marginal probabilities $p(x_t, x_{t+1} | y)$ and $p(x_t | y)$. In the HMM context, this algorithm is often referred to as the *forward-backward algorithm* [196]. These marginal probabilities are often of interest in and of themselves, but are also important in their role as expected sufficient statistics in an expectation-maximization (EM) algorithm for estimating the parameters of the HMM. Similarly, the maximum *a posteriori* state sequence can also be computed by the junction tree algorithm (with summation replaced by maximization) — in the HMM context the resulting algorithm is generally referred to as the *Viterbi algorithm* [82].

Gene-finding provides an important example of the application of the HMM [72]. To a first order of approximation, the genomic sequence of an organism can be segmented into regions containing genes and intergenic regions (that separate the genes), where a gene is defined as a sequence of nucleotides that can be further segmented into meaningful intragenic structures (exons and introns). The boundaries between these segments are highly stochastic and hence difficult to find reliably.

Hidden Markov models have been the methodology of choice for attacking this problem, with designers choosing states and state transitions to reflect biological knowledge concerning gene structure [44]. Hidden Markov models are also used to model certain aspects of protein structure. For example, membrane proteins are specific kinds of proteins that embed themselves in the membranes of cells, and play important roles in the transmission of signals in and out of the cell. These proteins loop in and out of the membrane many times, alternating between hydrophilic amino acids (which prefer the environment of the membrane) and hydrophobic amino acids (which prefer the environment inside or outside the membrane). These and other biological facts are used to design the states and state transition matrix of the *trans-membrane hidden Markov model*, an HMM for modeling membrane proteins [141].

Tree-structured models also play an important role in bioinformatics and language processing. For example, phylogenetic trees can be treated as graphical models. As shown in Figure 2.4(b), a phylogenetic tree is a tree-structured graphical model in which a set of observed nucleotides (or other biological characters) are assumed to have evolved from an underlying set of ancestral species. The conditional probabilities in the tree are obtained from evolutionary substitution models, and the computation of likelihoods are achieved by a recursion on the tree known as “pruning” [79]. This recursion is a special case of the junction tree algorithm.

Figure 2.5 provides examples of more complex graphical models that have been explored in bioinformatics. Figure 2.5(a) shows a *hidden*

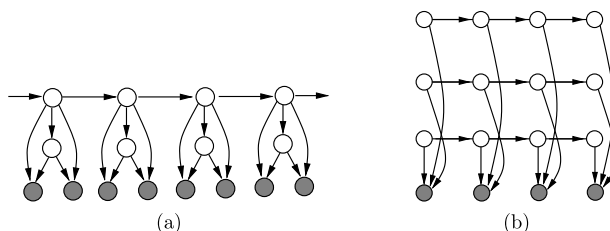


Fig. 2.5 Variations on the HMM used in bioinformatics. (a) A phylogenetic HMM. (b) The factorial HMM.

Markov phylogeny, an HMM in which the observations are sets of nucleotides related by a phylogenetic tree [165, 193, 218]. This model has proven useful for gene-finding in the human genome based on data for multiple primate species [165]. Figure 2.5(b) shows a *factorial HMM*, in which multiple chains are coupled by their links to a common set of observed variables [92]. This model captures the problem of *multi-locus linkage analysis* in genetics, where the state variables correspond to phase (maternal or paternal) along the chromosomes in meiosis [232].

2.4.5 Language and speech processing

In language problems, HMMs also play a fundamental role. An example is the part-of-speech problem, in which words in sentences are to be labeled as to their part of speech (noun, verb, adjective, etc.). Here the state variables are the parts of speech, and the transition matrix is estimated from a corpus via the EM algorithm [163]. The result of running the Viterbi algorithm on a new sentence is a tagging of the sentence according to the hypothesized parts of speech of the words in the sentence. Moreover, essentially all modern speech recognition systems are built on the foundation of HMMs [117]. In this case the observations are generally a sequence of short-range speech spectra, and the states correspond to longer-range units of speech such as phonemes or pairs of phonemes. Large-scale systems are built by composing elementary HMMs into larger graphical models.

The graphical model shown in Figure 2.6(a) is a *coupled HMM*, in which two chains of state variables are coupled via links between the

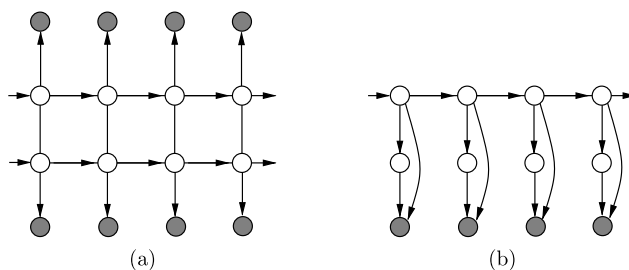


Fig. 2.6 Extensions of HMMs used in language and speech processing. (a) The coupled HMM. (b) An HMM with mixture-model emissions.

chains; this model is appropriate for fusing pairs of data streams such as audio and lip-reading data in speech recognition [208]. Figure 2.6(b) shows an HMM variant in which the state-dependent observation distribution is a finite mixture model. This variant is widely used in speech recognition systems [117].

Another model class that is widely studied in language processing are so-called “bag-of-words” models, which are of particular interest for modeling large-scale corpora of text documents. The terminology bag-of-words means that the order of words in a document is ignored — i.e., an assumption of exchangeability is made. The goal of such models is often that of finding latent “topics” in the corpus, and using these topics to cluster or classify the documents. An example bag-of-words model is the *latent Dirichlet allocation* model [32], in which a topic defines a probability distribution on words, and a document defines a probability distribution on topics. In particular, as shown in Figure 2.7, each document in a corpus is assumed to be generated by sampling a Dirichlet variable with hyperparameter α , and then repeatedly selecting a topic according to these Dirichlet probabilities, and choosing a word from the distribution associated with the selected topic.²

2.4.6 Image Processing and Spatial Statistics

For several decades, undirected graphical models or Markov random fields have played an important role in image processing

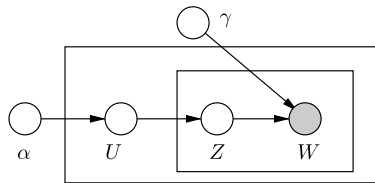


Fig. 2.7 Graphical illustration of the latent Dirichlet allocation model. The variable U , which is distributed as Dirichlet with parameter α , specifies the parameter for the multinomial “topic” variable Z . The “word” variable W is also multinomial conditioned on Z , with γ specifying the word probabilities associated with each topic. The rectangles, known as *plates*, denote conditionally independent replications of the random variables inside the rectangle.

²This model is discussed in more detail in Example 3.5 of Section 3.3.

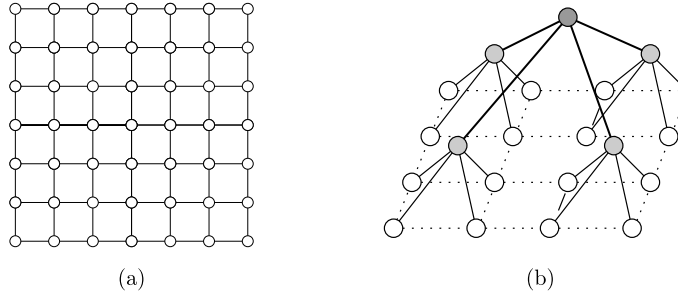


Fig. 2.8 (a) The 4-nearest neighbor lattice model in 2D is often used for image modeling. (b) A multiscale quad tree approximation to a 2D lattice model [262]. Nodes in the original lattice (drawn in white) lie at the finest scale of the tree. The middle and top scales of the tree consist of auxiliary nodes (drawn in gray), introduced to model the fine scale behavior.

[e.g., 58, 89, 107, 263], as well as in spatial statistics more generally [25, 26, 27, 201]. For modeling an image, the simplest use of a Markov random field model is in the pixel domain, where each pixel in the image is associated with a vertex in an underlying graph. More structured models are based on feature vectors at each spatial location, where each feature could be a linear multiscale filter (e.g., a wavelet), or a more complicated nonlinear operator.

For image modeling, one very natural choice of graphical structure is a 2D lattice, such as the 4-nearest neighbor variety shown in Figure 2.8(a). The potential functions on the edges between adjacent pixels (or more generally, features) are typically chosen to enforce local smoothness conditions. Various tasks in image processing, including denoising, segmentation, and super-resolution, require solving an inference problem on such a Markov random field. However, exact inference for large-scale lattice models is intractable, which necessitates the use of approximate methods. Markov chain Monte Carlo methods are often used [89], but they can be too slow and computationally intensive for many applications. More recently, message-passing algorithms such as sum-product and tree-reweighted max-product have become popular as an approximate inference method for image processing and computer vision problems [e.g., 84, 85, 137, 168, 227].

An alternative strategy is to sidestep the intractability of the lattice model by replacing it with a simpler — albeit approximate — model.

For instance, multiscale quad trees, such as that illustrated in Figure 2.8(b), can be used to approximate lattice models [262]. The advantage of such a multiscale model is in permitting the application of efficient tree algorithms to perform exact inference. The trade-off is that the model is imperfect, and can introduce artifacts into image reconstructions.

2.4.7 Error-Correcting Coding

A central problem in communication theory is that of transmitting information, represented as a sequence of bits, from one point to another. Examples include transmission from a personal computer over a network, or from a satellite to a ground position. If the communication channel is noisy, then some of the transmitted bits may be corrupted. In order to combat this noisiness, a natural strategy is to add redundancy to the transmitted bits, thereby defining codewords. In principle, this coding strategy allows the transmission to be decoded perfectly even in the presence of some number of errors.

Many of the best codes in use today, including turbo codes and low-density parity check codes [e.g., 87, 166], are based on graphical models. Figure 2.9(a) provides an illustration of a very small parity check code, represented here in the factor graph formalism [142]. (A somewhat larger code is shown in Figure 2.9(b)). On the left, the six white nodes represent the bits x_i that comprise the codewords (i.e., binary sequences of length six), whereas the gray nodes represent noisy observations y_i associated with these bits. On the right, each of the black square nodes corresponds to a factor ψ_{stu} that represents the parity of the triple $\{x_s, x_t, x_u\}$. This parity relation, expressed mathematically as $x_s \oplus x_t \oplus x_u \equiv z_{stu}$ in modulo two arithmetic, can be represented as an undirected graphical model using a compatibility function of the form:

$$\psi_{stu}(x_s, x_t, x_u) := \begin{cases} 1 & \text{if } x_s \oplus x_t \oplus x_u = 1 \\ 0 & \text{otherwise.} \end{cases}$$

For the code shown in Figure 2.9, the parity checks range over the set of triples $\{1, 3, 4\}$, $\{1, 3, 5\}$, $\{2, 4, 6\}$, and $\{2, 5, 6\}$.

The decoding problem entails estimating which codeword was transmitted on the basis of a vector $y = (y_1, y_2, \dots, y_m)$ of noisy observations.

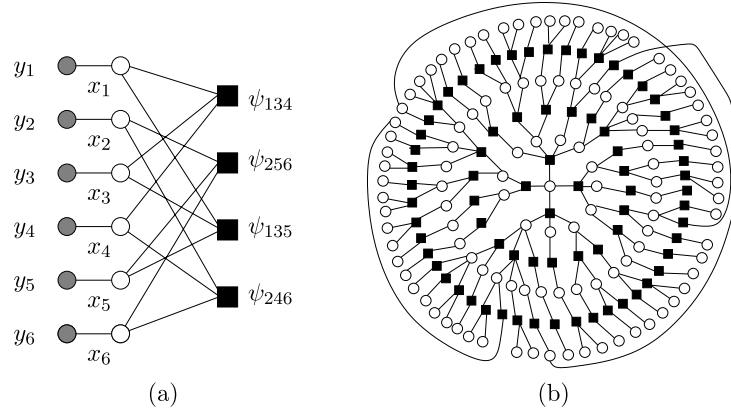


Fig. 2.9 (a) A factor graph representation of a parity check code of length $m = 6$. On the left, circular white nodes $\bigcirc$ represent the unobserved bits x_i that define the code, whereas circular gray nodes represent the observed values y_i , received from the channel. On the right, black squares $\blacksquare$ represent the associated factors, or parity checks. This particular code is a $(2,3)$ code, since each bit is connected to two parity variables, and each parity relation involves three bits. (b) A large factor graph with a “locally tree-like” structure. Random constructions of factor graphs on m vertices with bounded degree have cycles of typical length $\propto \log m$; this tree-like property is essential to the success of the sum-product algorithm for approximate decoding [200, 169].

With the specification of a model for channel noise, this decoding problem can be cast as an inference problem. Depending on the loss function, optimal decoding is based either on computing the marginal probability $p(x_s = 1 \mid y)$ at each node, or computing the most likely codeword (i.e., the mode of the posterior). For the simple code of Figure 2.9(a), optimal decoding is easily achievable via the junction tree algorithm. Of interest in many applications, however, are much larger codes in which the number of bits is easily several thousand. The graphs underlying these codes are not of low treewidth, so that the junction tree algorithm is not viable. Moreover, MCMC algorithms have not been deployed successfully in this domain.

For many graphical codes, the most successful decoder is based on applying the sum-product algorithm, described in Section 2.6. Since the graphical models defining good codes invariably have cycles, the sum-product algorithm is not guaranteed to compute the correct marginals, nor even to converge. Nonetheless, the behavior of this

approximate decoding algorithm is remarkably good for a large class of codes. The behavior of sum-product algorithm is well understood in the asymptotic limit (as the code length m goes to infinity), where martingale arguments can be used to prove concentration results [160, 199]. For intermediate code lengths, in contrast, its behavior is not as well understood.

2.5 Exact Inference Algorithms

In this section, we turn to a description of the basic exact inference algorithms for graphical models. In computing a marginal probability, we must sum or integrate the joint probability distribution over one or more variables. We can perform this computation as a sequence of operations by choosing a specific ordering of the variables (and making an appeal to Fubini's theorem). Recall that for either directed or undirected graphical models, the joint probability is a factored expression over subsets of the variables. Consequently, we can make use of the distributive law to move individual sums or integrals across factors that do not involve the variables being summed or integrated over. The phrase “exact inference” refers to the (essentially symbolic) problem of organizing this sequential computation, including managing the intermediate factors that arise. Assuming that each individual sum or integral is performed exactly, then the overall algorithm yields an exact numerical result.

In order to obtain the marginal probability distribution of a single variable X_s , it suffices to choose a specific ordering of the remaining variables and to eliminate — that is, sum or integrate out — variables according to that order. Repeating this operation for each individual variable would yield the full set of marginals. However, this approach is wasteful because it neglects to share intermediate terms in the individual computations. The sum-product and junction tree algorithms are essentially dynamic programming algorithms based on a calculus for sharing intermediate terms. The algorithms involve “message-passing” operations on graphs, where the messages are exactly these shared intermediate terms. Upon convergence of the algorithms, we obtain marginal probabilities for all cliques of the original graph.